

KLASSYGO LLP

HRM PLATFORM - TECHNICAL SPRINT PLAN

Day 1 to Day 14 | Build. Commit. Ship & Repeat.

The world has Workday

KLASSYGO HRM makes all of them look like prototypes.

Built in 12 days. By this team.

Issued by	Internship Lead, KLASSYGO LLP
Sprint	Day 1 to Day 12 Build Day 13 to Day 14 Board Review
Submission	GitHub commit + screen recording every day by 6:00 PM
Standup Reviews	Day 3 and Day 9 - live screen demo, no slides
Stack	React + Vite + Tailwind + Node + Express + MongoDB + Socket.io + AI
Deployment	Entirely Local via VS Code (Local Network Only)

WEEK 1 - DAYS 1 TO 6 | VIBE CODE → VS CODE → FIRST LIVING MODULE

Week 1 is the foundation week. Every person moves from vibe coded prototype to real working code in VS Code. By Day 6 your module has real data, real routes and real UI. Nothing hardcoded.

DAY 1 - ENVIRONMENT SETUP + VIBE CODE YOUR FULL MODULE

9:00 AM - 1:00 PM

Project Setup and Vibe Coding

- Fork the KLASSYGO HRM GitHub organisation repo. Clone it locally. Set up your branch: feature/your-module-name. Never push to main directly.
- GitHub Docs - Branching: <https://docs.github.com/en/get-started/quickstart/github-flow>
- Install all dependencies: Node.js, npm, Vite, Tailwind, MongoDB Compass, VS Code extensions (Live Server, Thunder Client, GitLens, MongoDB for VS Code, ES7 Snippets).
- Open Lovable or Bolt or v0. Describe your entire module in one detailed prompt. (more

2:00 PM - 6:00 PM

Backend Schema Design

- Backend P1: Design the full MongoDB schema. Collections: users, interns, batches, tasks, standups, attendance, announcements, certificates, notifications. Every field named and typed before a single line of backend code is written.
- Mongoose Schema Guide: <https://mongoosejs.com/docs/guide.html>
- Backend P2: Map every API endpoint the frontend will need. Method, route, request body, response shape. Document in a shared Google Sheet before writing any code.
- All four: set up the project folder structure. Frontend and backend in separate repos. Agree on the base API URL format: /api/v1/resource.

like 3D, more like live not static things in it)
Include: layout, colour scheme (Deep Black #0A0A0A, Ocean Blue #1A7BC4, Golden Yellow #FFB800) all screens and all user interactions.

- Export the vibe coded output. Read every file. Annotate anything you do not understand - those are your Day 2 morning questions.
- Frontend **P1**: Build login, lead dashboard shell and intern profile card layout. Frontend **P2**: Build intern home screen, standup form and kanban board shell.

- Backend **P1**: Set up Express server, connect to local MongoDB, confirm connection logs in terminal.
- Express Getting Started:
<https://expressjs.com/en/starter/installing.html>
- Backend **P2**: Install all backend dependencies: express, mongoose, dotenv, cors, bcrypt, jsonwebtoken, socket.io, multer, node-cron, pdfkit, express-rate-limit, nodemon.

End of Day Submission - GitHub commit with vibe coded prototype files + screen recording walking through every screen of your module. Minimum 3 minutes.

DAY 2 - AUTH SYSTEM + LOGIN UI

9:00 AM - 1:00 PM

Frontend - Login and Signup UI

- Frontend **P1**: Build the cinematic login page. Deep black background. Animated gradient behind a glassmorphism card. KLASSYGO left top. Email and password fields. Role selector: Lead or Intern. Submit button with loading spinner state.
- Framer Motion - Animation:
<https://www.framer.com/motion/>
- Tailwind Glassmorphism - backdrop-blur:
<https://tailwindcss.com/docs/backdrop-blur>
- Frontend **P1**: Build form validation - empty field error, invalid email format error, password under 8 characters error. Every error shown inline under the field not in a pop-up.
- Frontend **P2**: Build the skeleton loading screen that shows while auth is processing. No white flash. No blank page. Animated placeholder blocks.
- Tailwind Animate:
<https://tailwindcss.com/docs/animation>
- Frontend **P2**: Build the post-login redirect logic. Lead goes to /dashboard/lead. Intern goes to /dashboard/intern. Role stored in localStorage on successful login.

2:00 PM - 6:00 PM

Backend - Auth APIs

- Backend **P1**: Build POST /api/v1/auth/register - accepts name, email, password, role. Hash password with bcrypt (12 salt rounds). Save user to MongoDB. Return JWT.
- bcrypt npm: <https://www.npmjs.com/package/bcrypt>
- jsonwebtoken npm:
<https://www.npmjs.com/package/jsonwebtoken>
- Backend **P1**: Build POST /api/v1/auth/login - verify email exists, compare password hash, return signed JWT with userId and role in payload. Token expiry: 7 days.
- Backend **P2**: Build auth middleware - verifyToken function that reads Authorization header, verifies JWT, attaches user to req.user. Every protected route uses this middleware.
- Backend **P2**: Build role middleware - requireLead and requireIntern functions. Lead trying to access intern-only route gets 403. Intern trying to access lead route gets 403.
- Backend **P1**: Add express-rate-limit to auth routes. Max 10 requests per 15 minutes per IP. Prevents brute force.
- express-rate-limit:
<https://www.npmjs.com/package/express-rate-limit>
- Test every auth endpoint in Thunder Client before connecting to frontend.

End of Day Submission - GitHub commit with working auth flow. Screen recording showing register, login as lead, login as intern and role-based redirect working end to end.

DAY 3 - STANDUP 1 - LIVE DEMO + INTERN PROFILES MODULE

9:00 AM - 1:00 PM

STANDUP 1 -

- Present your module prototype live. Lead reviews every screen.
- No slides. No descriptions. Running UI or working API in Thunder Client - that is your presentation.
- Lead gives structured feedback: what is right, what needs to change, what is missing entirely.
- Corrections noted. You have until 6:00 PM to apply every feedback point from standup.
- Frontend P1: Apply all login and dashboard UI feedback from standup immediately.
- Frontend P2: Apply all intern home screen and standup form feedback immediately.

2:00 PM - 6:00 PM

Intern Profile APIs + UI

- Backend P1: Build GET /api/v1/interns - returns all interns (lead only). Include: name, track, mentor, batch, startDate, taskCompletionRate, currentTask.
- Backend P1: Build GET /api/v1/interns/:id - full intern profile for lead. Same data plus standup history and attendance summary.
- Backend P1: Build POST /api/v1/interns - create a new intern profile (lead only). Assign to batch, track, mentor.
- Backend P2: Build PATCH /api/v1/interns/:id - update intern profile fields. Support partial updates - only changed fields sent in body.
- Frontend P1: Build the intern profile card component. Avatar circle, track badge (colour coded by track), progress ring showing task completion percentage, current task label, days remaining chip.
- Recharts - Progress Circle: <https://recharts.org/en-US/api/RadialBarChart>
- Frontend P1: Wire intern profile cards to GET /api/v1/interns. Real data from database. No hardcoded names.

End of Day Submission - GitHub commit with intern profile API working and frontend cards showing real database data. Screen recording demonstrating full flow.

DAY 4 - LEAD DASHBOARD - THE COMMAND CENTRE

9:00 AM - 1:00 PM

Frontend - Lead Dashboard Build

- Frontend P1: Build the lead dashboard layout. Left sidebar with navigation icons. Top bar with KLASSYGO logo and lead name. Main content area with three zones: summary stats, intern grid, live activity feed.
- Summary stats row: total interns, standups submitted today, tasks overdue, attendance rate this week. Each stat in a glassmorphism card with a subtle animated count-up on load.
- Intern grid: all intern profile cards from Day 3, filterable by track (UI/UX, Ops, Full Stack). Filter tabs at top of grid.

2:00 PM - 6:00 PM

Backend - Dashboard Stats APIs + Socket Setup

- Backend P1: Build GET /api/v1/dashboard/stats - returns: totalInterns, standupsToday, tasksOverdue, weeklyAttendanceRate. Computed from real collections, not stored as a separate document.
- Backend P1: Set up Socket.io server. Configure CORS to allow frontend origin. Emit a live-activity event whenever: standup submitted, task status changed, announcement published.
- Socket.io Server Setup: <https://socket.io/docs/v4/server-installation/>
- Backend P2: Build GET /api/v1/dashboard/activity - returns last 20 activity events across all interns sorted by timestamp descending. Used as initial load before socket takes over.

- Framer Motion Layout Animations: <https://www.framer.com/motion/layout-animations/>
- Frontend P2: Build the live activity feed panel on the right side of the dashboard. Shows real time events: intern submitted standup, task marked complete, leave request raised. Each event as a timestamped card.
- Frontend P2: Add keyboard shortcuts to the lead dashboard. Press I to go to interns list. Press T to go to tasks. Press A to go to attendance. Show a keyboard shortcut hint modal when lead presses?

- Frontend P2: Connect Socket.io client to backend. On live-activity event, prepend new activity card to the feed with a slide-in animation. Feed updates live without any refresh.
- Socket.io Client: <https://socket.io/docs/v4/client-installation/>
- Backend P2: Test socket connection end to end. Submit a test standup via Thunder Client and confirm the activity event appears on the frontend in real time.

End of Day Submission - GitHub commit with lead dashboard showing real stats and live activity feed updating via Socket.io. Screen recording showing real time event appearing without page refresh.

DAY 5 - TASK TRACKER + KANBAN BOARD

9:00 AM - 1:00 PM

Frontend - Kanban Board Build

- Frontend P2: Build the Kanban task board with four columns: Not Started, In Progress, Under Review, Done. Each column has a count badge showing how many tasks are in it.
- Implement drag and drop using dnd-kit. Cards must animate smoothly when dragged between columns. Column count updates instantly when a card moves.
- dnd-kit - Drag and Drop: <https://dndkit.com/>
- Each task card shows: task title, assigned intern avatar, due date, priority badge (High / Medium / Low colour coded), blocker flag (red exclamation if intern flagged a blocker).
- Frontend P2: Build task detail modal. Click any card to open a full detail view: task description, intern assigned, standup history linked to this task, lead comment field, status change dropdown.
- Frontend P1: Add task board to the lead dashboard as a tab. Lead can switch between Intern Grid view and Task Board view without leaving the dashboard.
- Framer Motion AnimatePresence for tab transitions: <https://www.framer.com/motion/animate-presence/>

2:00 PM - 6:00 PM

Backend - Task APIs

- Backend P2: Build POST /api/v1/tasks - create task with title, description, assignedTo (intern ID), dueDate, priority, status (defaults to Not Started).
- Backend P2: Build GET /api/v1/tasks - returns all tasks grouped by status for Kanban. Lead gets all tasks. Intern gets only tasks assigned to them.
- Backend P2: Build PATCH /api/v1/tasks/:id/status - update task status. Emit socket event task-status-changed with taskId and newStatus so lead dashboard updates live.
- Backend P2: Build PATCH /api/v1/tasks/:id/blocker - intern flags a blocker with a description. Sets blocker: true and blockerNote on the task. Lead sees red flag on card immediately via socket.
- Backend P1: Build GET /api/v1/tasks/analytics - returns: completionRate per intern, averageTimePerStatus, overdueTasks count. Used for the analytics dashboard on Day 8.
- Test all task endpoints in Thunder Client. Verify socket events fire correctly on status change and blocker flag.

End of Day Submission - GitHub commit with fully working Kanban board. Drag a card between columns and confirm status updates in database. Screen recording showing drag and drop and blocker flag in action.

DAY 6 - STANDUP SYSTEM + ATTENDANCE MODULE

9:00 AM - 1:00 PM

Frontend - Standup Form + Attendance UI

- Frontend P2: Build the standup submission form for the intern view. Fields: what did you complete today, what are you working on tomorrow, any blockers (yes/no toggle - if yes show a text field), mood indicator (5 emoji options from burnt out to energised), task completion percentage slider (0 to 100).
- Form auto-saves to localStorage every 30 seconds. If intern closes the tab and reopens the form, draft is restored. Show a Saved draft badge with timestamp.
- On submit, show a full screen success animation (Framer Motion) before redirecting back to intern home.
- Frontend P2: Build the attendance calendar heatmap. Styled like a GitHub contribution graph. Each day is a coloured square: green for present, red for absent, yellow for late, blue for approved leave, grey for future.
- GitHub contribution graph style inspiration: <https://docs.github.com/en/account-and-profile/setting-up-and-managing-your-github-profile/managing-contribution-settings-on-your-profile/why-are-my-contributions-not-showing-up-on-my-profile>
- Frontend P1: Build the leave request modal for interns. Fields: leave start date, leave end date, reason (mandatory free text, submit blocked without it). Client-side check blocks submission with an inline error - "Leave must be applied at least 1 day in advance" - if the selected start date is today or earlier. On submit, show status as Pending; update to Approved or Rejected the moment the admin board acts, with no page refresh needed.
- Frontend P1: Build the Leave Requests panel on the admin board (lead dashboard). Table of every request: intern name, dates, reason, days requested, status. Approve and Reject buttons on each pending row - one click, no separate page. A notification bell in the top bar shows a live unread-count badge that increments the instant a new leave request comes in, and the request also appears in the live activity feed from Day 4.

2:00 PM - 6:00 PM

Backend - Standup + Attendance APIs

- Backend P1: Build POST /api/v1/standups - intern submits standup. Validate: one standup per intern per day (return 409 if duplicate). Save with timestamp and intern ID.
- Backend P1: Build GET /api/v1/standups/:internId - returns all standups for an intern sorted by date. Used in the intern profile and task detail modal.
- Backend P2: Build POST /api/v1/attendance/mark - mark attendance for an intern. Status options: present, absent, late, leave. Validate: cannot mark future dates.
- Backend P2: Build GET /api/v1/attendance/:internId - returns full attendance array for the heatmap. Each entry: date and status.
- Backend P2: Build POST /api/v1/attendance/leave-request - intern submits fromDate, toDate and reason. Reason is required - return 400 if missing. Server-side validate fromDate is at least 1 calendar day ahead of the current date - return 400 with "Leave must be applied at least 1 day before the leave date" if violated, so the rule cannot be bypassed by editing the frontend. Save with status: pending. Emit a socket event leave-request-raised (intern name, dates, reason) and write a notifications document targeted at the admin board so the request is not missed even if no one is online when it fires.
- Backend P2: Build PATCH /api/v1/attendance/leave-request/:id/approve and PATCH /api/v1/attendance/leave-request/:id/reject - admin board approves or rejects a pending request (lead only). On approve, mark the corresponding attendance dates as leave. Emit a socket event leave-status-updated targeted at the requesting intern's userId and write a notifications document for that intern - "Your leave request for [dates] has been approved/rejected" - so they are notified the moment a decision is made, even if they were offline at the time.
- Backend P1: Add node-cron job - every day at midnight automatically mark absent any intern who has not submitted attendance for that day.
- node-cron: <https://www.npmjs.com/package/node-cron>

End of Day Submission - GitHub commit with standup form submitting to database and attendance heatmap showing real data. Screen recording of full standup flow and calendar updating.

WEEK 2 - DAYS 7 TO 12 | AI LAYER + INTEGRATION + DEPLOYMENT

Week 2 is where KLASSYGO HRM stops being a project and becomes a product. AI layer goes in. Every module connects. The system deploys. Board-ready by Day 12.

DAY 7 - AI PERFORMANCE ENGINE + CERTIFICATE GENERATOR

9:00 AM - 1:00 PM

Backend - AI Weekly Summary Engine

- Backend P1: Integrate OpenAI or Gemini API. Build a function `generatePerformanceSummary(internId)` that pulls the last 7 standup logs for an intern and sends them to the AI with a structured prompt: Analyse these standup logs and write a 3 sentence performance summary covering progress made, patterns observed and one constructive recommendation. Tone: direct and professional.
- OpenAI Node SDK: <https://platform.openai.com/docs/libraries/node-js-library>
- Google Gemini API Node Quickstart: <https://ai.google.dev/gemini-api/docs/quickstart?lang=node>
- Backend P1: Build a node-cron job that runs every Sunday at 11:59 PM. Iterates through all active interns. Calls `generatePerformanceSummary` for each. Stores the result in the intern document as `weeklyAISummary` with a `generatedAt` timestamp.
- Backend P1: Build GET `/api/v1/interns/:id/ai-summary` — returns the latest AI summary for an intern. Lead sees this on the intern profile page.
- Frontend P1: Add AI Summary card to the intern profile page. Shows the weekly AI paragraph with a Generated by KLASSYGO Intelligence badge and the date it was generated.
- Add a Regenerate Summary button (lead only) that calls the AI endpoint on demand for any intern.

2:00 PM - 6:00 PM

Backend –

Certificate and Offer Letter Generator

- Backend P2: Build the PDF certificate generator using PDFKit. Template: upon submitting the intern username and password they should be able to know whether they are eligible or not, this depends on how they performed, did they meet all the criteria, to know that the intern can submit request based on request the lead will check and update whether they are eligible or not in less than 48hrs and if eligible the status gets updated so that they can directly download their file.
- PDFKit Node: <https://pdfkit.org/>
- Backend P2: Build GET `/api/v1/certificates/generate/:internId` - generates the certificate PDF server side and streams it directly to the browser as a downloadable file. No frontend PDF library. Pure server-side generation.
- Backend P2: Build the same system for offer letters using a different template. POST `/api/v1/offers/generate` - accepts `internId` and `startDate`, generates the offer letter PDF.
- Frontend P1: Add Download Certificate button to the intern profile (lead only, appears only when internship status is Completed). Single click downloads the generated PDF.
- Frontend P2: Add Download Offer Letter button to the onboarding checklist screen for new interns.
- Test both PDF endpoints in Thunder Client. Download the file and verify layout, intern name and dates are correct.

End of Day Submission - GitHub commit with AI summary generating from real standup data and certificate PDF downloading correctly. Screen recording showing AI output and PDF download.

DAY 8 - ANALYTICS DASHBOARD + ONBOARDING FLOW

9:00 AM - 1:00 PM

Frontend - Analytics Dashboard

- Frontend P1: Build the Analytics tab on the lead dashboard. Four chart sections: intern completion rate bar chart (one bar per intern), weekly standup submission line chart (7-day trend), track performance comparison radar chart, attendance rate donut chart per intern.
- Recharts - All chart types: <https://recharts.org/en-US/guide>
- All charts pull from the analytics APIs built by Backend P1 on Day 5. No static data anywhere.
- Frontend P1: Add a batch selector dropdown at the top of analytics. Lead can switch between Batch 2025 and Batch 2026 and upcoming batches as well and all charts update instantly.
- Frontend P1: Add CSV export button on the analytics page. One click downloads the full attendance and performance data as a spreadsheet.
- Frontend P2: Add a performance leaderboard to the analytics tab. Ranked list of interns by overall score. Score computed as weighted average of standup completion rate, task completion rate and attendance rate.

2:00 PM - 6:00 PM

Frontend - Intern Onboarding Flow

- Frontend P2: Build the Day 1 onboarding flow for new interns. When an intern logs in for the first time (firstLogin: true flag in their profile), they see a guided onboarding checklist instead of the normal dashboard.
- Onboarding checklist items: Complete your profile, Read the KLASSYGO Brand Voice Guide, Review your 14-day task plan, Submit your first standup, Mark your attendance. Each item has a checkbox. Progress bar at top shows completion.
- When all five items are checked, trigger a confetti animation (use canvas-confetti) and a Welcome to KLASSYGO card before redirecting to the normal intern dashboard.
- canvas-confetti: <https://www.npmjs.com/package/canvas-confetti>
- Backend P1: Add firstLogin field to user schema. Set to true on register. Build PATCH /api/v1/users/:id/complete-onboarding that sets firstLogin to false. Called when intern completes all checklist items.
- Backend P2: Build GET /api/v1/interns/:id/onboarding-status — returns which checklist items are completed based on real data: has profile photo, has submitted standup, has marked attendance.

End of Day Submission - GitHub commit with analytics dashboard showing real chart data and onboarding flow triggering confetti on completion. Screen recording demonstrating both.

DAY 9 - STANDUP 2 - FULL SYSTEM INTEGRATION DEMO

9:00 AM - 1:00 PM

STANDUP 2 - Full System Live Demo

- This is not a module demo. This is the full integrated system running live.
- Someone logs in as lead. Someone logs in as intern on a different device on the same network.
- Lead dashboard shows real intern data. Intern submits a standup live - lead activity feed updates in real time without refresh.
- Lead drags a task to Done on the Kanban board. Intern dashboard shows task as completed instantly via socket.
- Lead views AI summary for an intern. Lead downloads a certificate. Intern completes onboarding flow and confetti fires.

2:00 PM - 6:00 PM

Post-Standup - Bug Fixes and Polish

- Apply every bug and gap flagged in Standup 2 immediately. This afternoon is for fixes only.
- Frontend P1: Fix all lead dashboard and profile page issues flagged.
- Frontend P2: Fix all intern view, standup form and kanban issues flagged.
- Backend P1: Fix all auth, profile, AI and socket issues flagged.
- Backend P2: Fix all task, attendance, standup and PDF issues flagged.

- Every feature demonstrated. No slides. No descriptions. Live system only.
- Lead gives final feedback. All bugs and gaps documented. Fixes assigned to specific people.

- By end of Day 9: every bug from standup is resolved and committed. No pending fixes carried into deployment week.
- Run the full demo flow again at 5:00 PM to confirm all fixes work. If anything still breaks, fix it before committing.

End of Day Submission - GitHub commit with all standup feedback applied. Screen recording of the complete fixed system running end to end.

DAY 10 - DATABASE + BACKEND LIVE VIA VS CODE

9:00 AM - 1:00 PM

Local MongoDB + Backend Environment Setup

- Backend P1: Confirm local MongoDB (set up on Day 1) is running. Open MongoDB Compass, connect to mongodb://localhost:27017, create the klassygo-hrm database if it does not already exist, and confirm every collection from the Day 1 schema is visible.
- MongoDB Compass Docs: <https://www.mongodb.com/docs/compass/current/>
- Backend P1: Confirm MONGODB_URI in .env is mongodb://localhost:27017/klassygo-hrm. No external connection string, no internet dependency - the database lives on the same machine as the backend.
- Backend P2: Create .env file with all environment variables: MONGODB_URI, JWT_SECRET, PORT, OPENAI_API_KEY (or GEMINI_API_KEY), FRONTEND_URL (the local Network URL from Day 11).
- Create .env.example with same keys, no values. Add .env to .gitignore. Push backend to GitHub.
- Backend P2: Update server.js to bind to 0.0.0.0. Add the network IP logger. Update CORS to allow both localhost and the local network frontend URL (the IP shown in the VS Code terminal).
- CORS in Express: <https://expressjs.com/en/resources/middleware/cors.html>

2:00 PM - 6:00 PM

Backend Live via VS Code (Local)

- Backend P1: In the VS Code terminal, run npm install then npm run dev (nodemon, auto-restarts on file changes) or npm start. Confirm the terminal logs MongoDB connected and prints both the Local and Network address the server is listening on.
- nodemon — auto-restart on file changes: <https://www.npmjs.com/package/nodemon>
- Double check every variable is filled in correctly in the local .env file: MONGODB_URI, JWT_SECRET, PORT, OPENAI_API_KEY, FRONTEND_URL. Nothing to configure on any external dashboard the backend reads everything from disk.
- Copy the Network URL printed in the VS Code terminal something like http://192.168.1.42:5000. This is the backend address the frontend, Thunder Client, and any test device on the WiFi will use.
- Backend P2: Test every API endpoint against the local Network URL using Thunder Client. Auth, interns, tasks, attendance, standups, PDF generation - every route confirmed working end to end.
- Backend P2: Test Socket.io connection to the locally running backend from a frontend on another device on the same network. Confirm real time events fire correctly across devices.
- Fix any environment variable issues or connection errors before EOD. Backend must be fully live and tested before Day 11.

End of Day Submission - GitHub commit with backend running locally via VS Code, reachable on the local network. Screen recording showing Thunder Client hitting the local Network API URL and getting correct responses.

DAY 11 - DEPLOYMENT - FRONTEND LIVE + FULL SYSTEM VIA VS CODE

9:00 AM - 1:00 PM

Frontend Production Prep

- Frontend P1: Replace any hardcoded localhost references with the backend's local Network URL from Day 10, set via environment variables. For Vite: VITE_API_URL in .env.production. For every fetch call: import.meta.env.VITE_API_URL.
- Vite Environment Variables: <https://vitejs.dev/guide/env-and-mode.html>
- Frontend P2: Do the same for Socket.io connection. Connect to the backend's Network URL, not localhost, so other devices on the WiFi can receive real-time events too.
- Frontend P1: Run npm run build locally. Fix any build errors. The dist folder must compile without errors before pushing to GitHub.
- Frontend P2: Check every page for console errors in production build. Fix all TypeScript or lint errors. Run Lighthouse on the local build - target 90+ performance score.
- Lighthouse in Chrome DevTools: <https://developer.chrome.com/docs/lighthouse/overview/>
- Both frontend: push final production-ready code to GitHub. Confirm all env variables are in .env.production and .env.example is committed.

2:00 PM - 6:00 PM

VS Code Deployment + Full System Test

- Frontend P1: Open the frontend project folder in VS Code. Run npm install if not already done. Confirm the Live Server extension installed on Day 1 is active. Add environment variable: VITE_API_URL = your backend's local Network URL from Day 10, in .env.production.
- Vite Preview — serve the production build locally: <https://vitejs.dev/guide/cli.html#vite-preview>
- In the VS Code terminal run npm run build, then npm run preview (or right-click dist/index.html and choose Open with Live Server). VS Code prints a Local URL and a Network URL in the terminal - something like http://192.168.1.42:4173 - the Network URL is what gets shared with anyone on the same WiFi.
- Frontend P2: Open the Network URL from VS Code on a phone, a laptop and a tablet, all connected to the same WiFi as the host laptop. Test every screen. Every button. Every form. Document any layout issues on mobile.
- All four: run the complete user journey end to end. Register as lead. Register as intern. Log in as both. Submit standup. Drag task. Get AI summary. Download certificate. Every feature tested on the VS Code-served frontend talking to the locally running backend.
- Fix any production-specific bugs immediately. These are usually CORS issues, missing env variables or hardcoded localhost references that slipped through.
- By end of Day 11: KLASSYGO HRM is fully running end to end, entirely through VS Code. Frontend served locally and reachable on the local network. Backend running locally on the same network. Database running locally via MongoDB Compass. Nothing hosted on any cloud platform.

End of Day Submission - GitHub commit with frontend running locally via VS Code, served on the local network. Screen recording opening the Network URL on a phone and walking through the complete system end to end.

DAY 12 - FINAL POLISH - BOARD-READY BY 6:00 PM

9:00 AM - 1:00 PM

Morning - Final QA Pass

- All four: spend the morning doing a complete quality assurance pass of the deployed system. Use this checklist.
- Auth: register new lead, register new intern, login both, verify role-based access blocks correctly, verify JWT expiry works.
- Intern view: complete onboarding flow, submit standup, drag task, request leave, download offer letter, check attendance heatmap.
- Lead view: view all intern cards, open an intern profile, view AI summary, approve leave request, drag task on kanban, view analytics charts, download certificate, publish announcement.
- Real time: confirm socket events fire on standup submit, task move and announcement publish. Confirm lead activity feed updates live.
- PDF: download certificate and offer letter. Open both. Verify intern name, dates and KLASSYGO branding are correct.
- Mobile: open the live URL on a phone. Every screen readable. No broken layouts. Forms functional on touch.

2:00 PM - 6:00 PM

Afternoon - Performance + Final Commit

- Frontend P1: Run Lighthouse on the locally served production build (via VS Code). Fix any critical performance issues: image optimisation, unused CSS purge, lazy loading for off-screen components.
- Frontend P2: Add a 404 page for unknown routes. Add a global error boundary in React that catches crashes and shows a clean error screen with a Return to Dashboard button.
- React Error Boundaries: <https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary>
- Backend P1: Add request logging using morgan. Every API hit logged with method, route and response time. Useful for board demo and future debugging.
- morgan npm: <https://www.npmjs.com/package/morgan>
- Backend P2: Add a GET /api/v1/health endpoint that returns system status: database connected, server uptime and current timestamp. Lead can hit this endpoint to confirm the backend is alive.
- All four: final commit to GitHub. Tag the release as v1.0.0. Write a clean README for both frontend and backend repos with setup instructions, env variable list and live URL.
- Send lead three things by 6:00 PM: live frontend URL, live backend URL and a 3-sentence summary of what your module does.

End of Day Submission - Final GitHub commit tagged v1.0.0. Screen recording of the complete live system running on the VS Code-served local build - full walkthrough as lead and as intern. This is your board demo recording.

DAYS 13 AND 14 - BOARD REVIEW

The build is done. It is deployed. It is live. Now it goes to the board.

Day 13 - Lead Review

- Lead conducts a full internal review of every module before it reaches the board. Every bug flagged gets fixed same day. No overnight pending.
- Role-based access verified: lead cannot access intern-only routes, intern cannot access lead routes.
- Data integrity check: all forms save correctly, all charts show accurate numbers, all PDFs generate with correct intern data.
- Mobile responsiveness confirmed on at least two different screen sizes.
- Fixes committed and running end to end through VS Code - frontend and backend both live on the local network — by 5:00 PM on Day 13.

Day 14 - Board Walkthrough

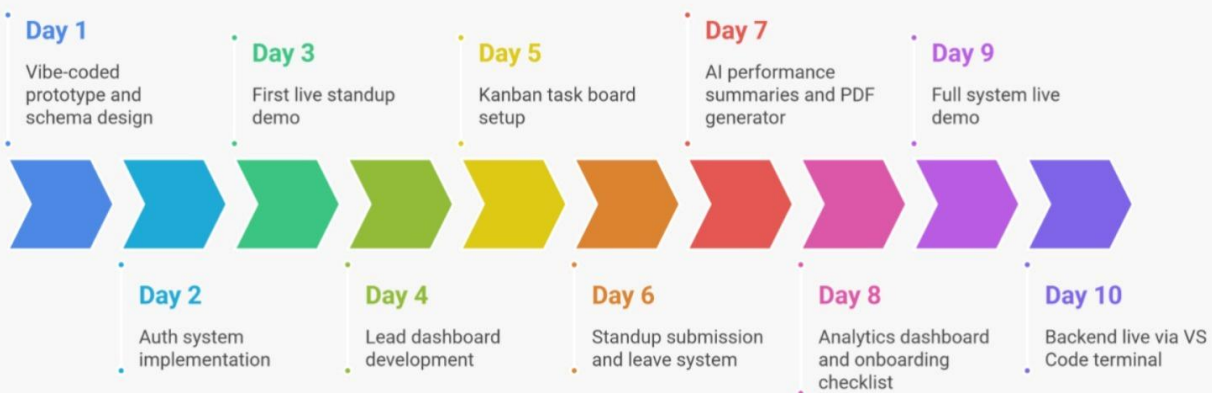
- Board opens the Network URL shown in the VS Code terminal — lead's laptop stays connected to the same WiFi as the board. No installs. No sharing screens. They access it themselves on their own device.
- Lead walks through the system as a lead user. Board walks through as an intern on their own device.
- Every question from the board gets a live demonstration not a verbal answer.
- Board assesses: does this solve the real KLASSYGO problem, is it usable by someone on Day 1 of their internship, can it scale to the next batch without rebuilding.
- You present what you built. You explain every decision. You own every screen.

FAANG/MANGO companies had years and hundreds of engineers.

You have 12 days and our strong team. Let's do this

- Internship Lead, KLASSYGO LLP

Project Development Timeline: From Prototype to Launch



How to deploy the application locally?

